

PKI, Magasin de Certificats et Signature de Scripts

Ce document aborde l'intégration de la cryptographie dans l'automatisation PowerShell. Dans un contexte DevOps ou d'Infrastructure as Code (IaC), la sécurité ne se gère plus manuellement via des interfaces graphiques (MMC). L'administrateur doit être capable d'auditer les certificats, d'en générer à la volée pour sécuriser des flux (WinRM, HTTPS), et de signer numériquement son code pour garantir la chaîne de confiance en production.

1. L'Abstraction du Magasin de Certificats : Le PSDrive Cert:

Le système d'exploitation Windows stocke ses clés cryptographiques et certificats dans un magasin sécurisé (Certificate Store). Traditionnellement, les administrateurs utilisent la console certlm.msc ou certmgr.msc pour interagir avec ce magasin.

PowerShell abstrait toute cette complexité grâce au **PSProvider Certificate**, qui expose le magasin sous la forme d'un système de fichiers virtuel : le **PSDrive Cert:**.

1.1 Architecture du lecteur Cert:

Le lecteur Cert: se divise en deux arborescences principales (les "ruches") :

- **Cert:\LocalMachine (Machine Locale)** : Certificats applicables au système d'exploitation entier, à tous les utilisateurs et aux services (ex: certificats IIS, WinRM, IPsec). Nécessite des privilèges Administrateur.
- **Cert:\CurrentUser (Utilisateur Courant)** : Certificats liés au profil de l'utilisateur connecté (ex: chiffrement EFS, signature de code personnelle, authentification client).

Au sein de ces ruches, les dossiers les plus critiques sont :

- \My : Magasin "Personnel" (certificats dont la machine possède la clé privée).
- \Root : Autorités de Certification (CA) de confiance.

1.2 Exploration et Audit en ligne de commande

Puisque Cert: est un lecteur logique, les commandes standards (Get-ChildItem, Where-Object) fonctionnent de manière transparente.

Lister tous les certificats personnels du serveur local

```
Get-ChildItem -Path Cert:\LocalMachine\My
```

Auditer la sécurité : Trouver tous les certificats expirant dans les 30 prochains jours

```
$DateLimite = (Get-Date).AddDays(30)
```

```
Get-ChildItem -Path Cert:\LocalMachine\My -Recurse |
```

```
Where-Object { $_.NotAfter -lt $DateLimite -and $_.NotAfter -gt (Get-Date) } |  
Select-Object Thumbprint, Subject, NotAfter
```

2. Génération de Certificats Auto-Signés (New-SelfSignedCertificate)

Pour des environnements de développement, de test (Sandboxing), ou pour chiffrer des communications internes (WinRM HTTPS), il est souvent nécessaire de générer des certificats à la volée.

La cmdlet New-SelfSignedCertificate permet de créer ces certificats sans avoir à déployer une infrastructure PKI (AD CS) complète.

2.1 Cas 1 : Certificat pour sécuriser un flux (TLS/HTTPS)

Création d'un certificat d'authentification serveur (identique à un certificat web) :

```
$CertWeb = New-SelfSignedCertificate -DnsName "srv-app01.formation.local", "srv-app01" `  
    -CertStoreLocation "Cert:\LocalMachine\My" `  
    -NotAfter (Get-Date).AddYears(1) `  
    -KeyAlgorithm RSA `  
    -KeyLength 2048
```

```
Write-Host "Certificat généré avec l'empreinte : (CertWeb.Thumbprint)"
```

2.2 Cas 2 : Certificat pour la signature de code (Code Signing)

Pour signer un script PowerShell, le certificat doit posséder une extension (EKU - Enhanced Key Usage) spécifique : *Code Signing*.

```
$CertSignature = New-SelfSignedCertificate -Subject "CN=DevOps Code Signing" `  
    -Type CodeSigningCert `  
    -CertStoreLocation "Cert:\CurrentUser\My"
```

```
Write-Host "Certificat de signature créé. Empreinte : (CertSignature.Thumbprint)"
```

3. Le Contrôle d'Exécution : ExecutionPolicy

Windows intègre un mécanisme de sécurité pour prévenir l'exécution accidentelle de scripts malveillants. Ce mécanisme ne remplace pas un antivirus, mais empêche le double-clic involontaire ou l'exécution de code téléchargé à la volée.

3.1 Les niveaux de politique

- **Restricted** : (Par défaut) Aucun script ne s'exécute. Seules les commandes interactives sont autorisées.
- **RemoteSigned** : Les scripts locaux s'exécutent. Les scripts téléchargés depuis Internet doivent être signés par un éditeur de confiance. (Standard pour les postes de développement).
- **AllSigned** : **TOUS** les scripts (même écrits localement) doivent être signés par une autorité de confiance. (Standard exigé sur les serveurs de production Tier 0 / Tier 1).
- **Bypass** : Aucune restriction. (Utilisé par la CI/CD ou les tâches automatisées pour exécuter un script ponctuel sans modifier le registre système : powershell.exe -ExecutionPolicy Bypass -File script.ps1).

Vérifier la politique actuelle
Get-ExecutionPolicy -List

Forcer une politique stricte sur la machine locale
Set-ExecutionPolicy -ExecutionPolicy AllSigned -Scope LocalMachine -Force

4. La Signature Numérique des Scripts (Authenticode)

En environnement de production (politique AllSigned), un script ne peut s'exécuter que s'il est muni d'une signature numérique (Authenticode).

Pourquoi signer un script ?

1. **Garantie d'identité** : Prouve cryptographiquement qui a écrit le script (L'équipe DevOps de l'entreprise).
2. **Garantie d'intégrité** : Si un attaquant (ou un ransomware) modifie ne serait-ce qu'une virgule dans le fichier .ps1, le hash cryptographique est brisé et Windows refusera formellement d'exécuter le script.

4.1 Processus de Signature d'un script (Set-AuthenticodeSignature)

1. **Récupération du certificat valide** :
Il faut d'abord isoler le certificat de "Code Signing" dans le magasin.
`$Cert = Get-Childitem -Path Cert:\CurrentUser\My -CodeSigningCert | Select-Object -First 1`
2. **Application de la signature sur le fichier** :
`Set-AuthenticodeSignature -FilePath "C:\Projets\InfraTools\InfraTools.psm1" -Certificate $Cert`
3. **Le Timestamping (Horodatage)** :
Si vous signez un script avec un certificat valable 1 an, la signature deviendra invalide

l'année suivante. Pour éviter cela, on utilise un serveur d'horodatage (Timestamp Server). Il certifie que le certificat était valide au moment précis de la signature, rendant la signature valable indéfiniment.

```
Set-AuthenticodeSignature -FilePath ".\script.ps1" `
    -Certificate $Cert `
    -TimestampServer "http://timestamp.digicert.com"
```

4.2 Vérification de l'intégrité (Get-AuthenticodeSignature)

Pour auditer un script avant exécution :

```
$Signature = Get-AuthenticodeSignature -FilePath "C:\Projets\InfraTools\InfraTools.psm1"
$Signature.Status # Retourne 'Valid' si intact, 'HashMismatch' si le fichier a été altéré
```

5. Synthèse : Pipeline de Publication

Dans une usine logicielle moderne, les développeurs ou les administrateurs ne signent jamais les scripts manuellement sur leurs postes de travail (leurs certificats n'ont pas de valeur en production).

Le flux de travail CI/CD standard (Security-as-Code) :

1. Le développeur écrit le code et pousse ses fichiers .psm1 et .psd1 non-signés sur Git.
2. Le pipeline CI/CD se déclenche.
3. Le pipeline lance **PSScriptAnalyzer** (Linter) et **Pester** (Tests).
4. Si tous les tests sont verts, l'orchestrateur (GitLab/Azure DevOps) extrait secrètement le certificat privé de l'entreprise depuis un coffre-fort (ex: Azure Key Vault).
5. Le pipeline exécute Set-AuthenticodeSignature sur l'ensemble du module.
6. Le module, désormais certifié "Production", est poussé sur les serveurs cibles ou publié dans un registre privé.